```
                    break;
                }
        } while (c!=F10);
}
void main(void) {
        thuchien();
}
```

3. CÁC CẤU TRÚC LỆNH ĐIỀU KHIỂN

3.1. Câu lệnh khối

Tập các câu lệnh được bao bởi hai dấu { . . . } được gọi là một câu lệnh khối. Về cú pháp, ta có thể đặt câu lệnh khối ở một vị trí bất kì trong chương trình. Tuy nhiên, nên đặt các câu lệnh khối ứng với các chu trình điều khiển lệnh như for, while, do . . while, if . . else, switch để hiển thị rõ cấu trúc của chương trình.

Ví dụ:

```
if (a > b ) {
câu_lệnh;
}
```

3.2. Cấu trúc lệnh if

Dạng 1:

```
if ( biểu thức)
        câu_lệnh;
```

Nếu biểu thức có giá trị đúng thì thực hiện câu_lệnh; Câu lệnh có thể hiểu là câu lệnh đơn hoặc câu lệnh khối, nếu câu lệnh là lệnh khối thì nó phải được bao trong { . . }.

Dạng 2:

```
if (biểu_thức)
        câu_lệnh_A;
else
        câu_lệnh_B;
```

Nếu biểu thức có giá trị đúng thì câu_lệnh_A sẽ được thực hiện, nếu biểu thức có giá trị sai thì câu_lệnh_B sẽ được thực hiện.

Dạng 3: Được sử dụng khi có nhiều lệnh if lồng nhau hoặc phải kiểm tra nhiều biểu thức khác nhau.

```
if (biểu_thức_1)
        câu_lệnh_1;
```

PTIT

```
else if (biểu_thức_2)
    câu_lệnh_2;
. . . . . .
else if (biểu_thức_k)
    Câu_lệnh_k;
else
câu_lệnh_k+1;
```

Nếu biểu_thức_i có giá trị đúng (0<i<=k) thì câu_lệnh_i sẽ được thực hiện, nếu không biểu thức nào có giá trị đúng thì câu_lệnh_k+1 sẽ được thực hiện.

Ví dụ: Tìm số lớn nhất trong hai số a và b:

```
#include <stdio.h>
void main(void){
float       a, b, max;
printf("\n Nhập a="); scanf("%f", &a); /* nhập giá trị cho biến a*/
printf("\n Nhập b="); scanf("%f", &b); /* nhập giá trị cho biến b*/
if (a>b) max=a;
else max= b;
printf("\n Max(a,b)=%6.2f",max);
getch();
}
```

Ghi chú:

Toán tử: &(tên_biến) lấy địa chỉ của biến. Câu lệnh scanf("%f",&a) có nghĩa là nhập một số thực vào địa chỉ ô nhớ dành cho biến a.

Ví dụ: Viết chương trình giải phương trình bậc 2 : $ax^2 + bx + c = 0$

```
#include    <stdio.h>
#include    <conio.h>
#include    <math.h>
void main(void){
    float   a, b, c, x1, x2, delta;
    clrscr();
    printf("\n Giải phương trình bậc 2:");
    /*đọc các hệ số a, b, c từ bàn phím");
    printf("\n Nhập hệ số a="); scanf("%f",&a);
    printf("\n Nhập hệ số b="); scanf("%f",&b);
    printf("\n Nhập hệ số c="); scanf("%f",&b);
    /* tính delta = b2 - 4ac*/
    delta=b*b-4*a*c;
    if (delta==0){
            printf("\n phương trình có 2 nghiệm kép x1=x2=%f", -b/(2*a));
    }
```

```
        else if(delta>0){
                printf("\n Phương trình có hai nghiệm");
                x1= ( -b + sqrt(delta) ) / (2*a);
                x1= ( -b - sqrt(delta) ) / (2*a);
                printf(" x1 = % 6.2f x2=%6.2f", x1,x2);
        }
        else {
                printf("\n Phương trình có nghiệm phức:");
                x1 = -b / (2 a); / phần thực */
                x2 = ( sqrt( -delta) ) / (2 * a);
                printf(" Real = %6.2f Im = % 6.2f", x1, x2);
        }
        getch();
}
```

3.3. Cấu *trúc lệnh switch*

Cấu trúc lệnh if thực hiện một phương án đúng trong hai phương án có thể xảy ra. Cấu trúc lệnh switch dùng để lựa chọn và thực hiện các phương án đúng có thể xảy ra.

Cú pháp

```
switch(biểu_thức_nguyên){
case hằng_nguyên_1: câu_lệnh_1; break;
case hằng_nguyên_2: câu_lệnh_2; break;
case hằng_nguyên_3: câu_lệnh_3; break;
. . . . . .
case hằng_nguyên_n: câu_lệnh_n; break;
default: câu_lệnh_n+1;break;
}
```

Thực hiện: Nếu biểu_thức_nguyên có giá trị trùng với hằng_nguyên_i thì câu_lệnh_i trở đi sẽ được thực hiện cho tới khi nào gặp từ khoá break để thoát khỏi chu trình.

Ví dụ: Nhập một số nguyên dương từ bàn phím và xác định xem số nguyên đó có phải là các số từ 1. .10 hay không? Trong trường hợp không phải là các số nguyên từ 1 . . 10 hãy đưa ra thông báo "số lớn hơn 10".

```
#include    <stdio.h>
#include    <conio.h>
void main(void){
    int n; clrscr();
    printf("\n Nhập n=");scanf("%d",&n);
    switch(n){
            case 1:
            case 2:
            case 3:
            case 4:
```

```
            case 5:
            case 6:
            case 7:
            case 8:
            case 9:
            case 10: printf("\n Số từ 1. .10"); break;
            default : printf("\n Số lớn hơn 10"); break;
        }
}
```

3.4. Vòng lặp for

Cú pháp:

```
for(biểu_thức_1; biểu_thức_2; biểu_thức_3)
        Câu_lệnh;
```

Câu_lệnh: Có thể là lệnh đơn hoặc lệnh khối, nếu là lệnh đơn thì câu lệnh trong thân chu trình for không cần thiết phải bao trong hai kí hiệu {, }. Nếu là lệnh khối (thân chu trình for có hơn một lệnh) thì nó phải được bao trong hai kí hiệu {, }.

Thực hiện:

Biểu_thức_1: Được gọi là biểu thức khởi đầu có nhiệm vụ khởi đầu các biến sử dụng trong chu trình, biểu_thức_1 chỉ được thực hiện duy nhất một lần khi bắt đầu bước vào chu trình. Nếu trong chu trình phải khởi đầu nhiều biểu thức thì mỗi biểu thức được phân biệt với nhau bởi một kí tự ','.

Biểu_thức_2: Được gọi là biểu thức kiểm tra và được thực hiện ngay sau khi thực hiện xong biểu_thức_1, nếu biểu thức kiểm tra có giá trị đúng (khác 0) thì câu lệnh trong thân của chu trình for sẽ được thực hiện, nếu biểu thức kiểm tra có giá trị sai thì điều khiển của chương trình chuyển về lệnh kế tiếp ngay sau thân của chu trình for.

Biểu_thức_3: Được gọi là biểu thức khởi đầu lại có nhiệm vụ khởi đầu lại các biến trong chu trình và được thực hiện ngay sau khi thực hiện xong câu_lệnh. Chu trình sẽ được lặp lại bằng việc thực hiện biểu thức kiểm tra.

Ví dụ: Viết chương trình in ra màn hình dãy các kí tự theo dạng sau:

```
A  B     C     D . . .Z
a  b     c     d . . .z
Z  Y     X     W. . .A
z  y     X     w. . .a
```

```
/* chương trình in dãy kí tự */
#include    <stdio.h>
void main(void){
char ch; clrscr();
for(ch ='A'; ch<='Z'; ch++)
```

```
 printf("%3c",ch);
printf("\n");
for(ch ='a'; ch<='z'; ch++)
 printf("%3c",ch);
printf("\n");
for(ch ='Z'; ch>='A'; ch--)
 printf("%3c",ch);
printf("\n");
for(ch ='z'; ch>='a'; ch--)
 printf("%3c",ch);
printf("\n");getch();
}
```

Ghi chú: Đối với ngôn ngữ C, kiểu dữ liệu char thực chất là một số nguyên có kích cỡ 1 byte, giá trị của byte là vị trí của kí tự trong bảng mã ASSCI. Do vậy, chương trình trên có thể viết lại bằng cách sau:

Ví dụ:

```
/* chương trình in dãy kí tự */
#include <stdio.h>
void main(void){
int ch; clrscr();
for(ch =65; ch<=90; ch++)
printf("%3c",ch);
printf("\n");
for(ch =97; ch<=122; ch++)
printf("%3c",ch);
printf("\n");
for(ch ='Z'; ch>='A'; ch--)
 printf("%3c",ch);
printf("\n");
for(ch ='z'; ch>='a'; ch--)
 printf("%3c",ch);
printf("\n");getch();
}
```

Ví dụ: Viết chương trình giải bài toán cổ "Trăm trâu trăm cỏ".

```
#include     <stdio.h>
#include     <conio.h>
void main(void) {
 unsigned int x, y, z; /* khai báo số trâu đứng, trâu nằm, trâu già*/
```

PTIT

```
    for( x=0;x<=20;x++){
      for(y=0;y<=33;y++){
        for(z=0;z<100;z+=3){
          if(x + y + z ==100 && (5*x + 3 *y + ( z / 3))==100){
            printf("\n Trâu đứng:%5d",x);
            printf(" Trâu nằm:%5d ",y);
            printf(" Trâu già:%5d", z);
          }
        }
      }
    }
}
```

3.5. Vòng lặp không xác định while

Cú pháp:

```
while(biểu_thức)
    câu_lệnh;
```

Trong khi biểu thức còn đúng thì câu lệnh sẽ được thực hiện, nếu biểu thức có giá trị sai điều khiển của chương trình chuyển về lệnh kế tiếp ngay sau thân của while. Nếu trong thân của while có nhiều hơn một lệnh thì nó phải được bao trong hai kí tự { . .}.

Ví dụ: Đếm số chữ số, khoảng trắng (space), dấu tab, dấu về đầu dòng và những kí tự khác được nhập từ bàn phím.

```
#include    <conio.h>
#include    <stdio.h>
#define     ESC 27      /* mã của phím ESC*/
#define     ENTER       13
void main(void){
    int     number=0, space=0, tab=0, enter=0, other=0;
    char ch;

clrscr();
    while( ( ch=getch() ) != ESC){ /* thực hiện nếu không phải là ESC*/
            if(ch>='0' && ch <='9')
                    number++;
            else if(ch ==' ')       space++;
            else if(ch =='\t')      tab++;
            else if(ch ==ENTER) enter ++;
            else other++;
    }
    printf("\n Số chữ số là: %d", number);
    printf("\n Số dấu trống là: %d", space);
```

```
        printf("\n Số dấu tab là: %d", tab);
        printf("\n Số dấu xuống dòng là: %d", enter);
        printf("\n Các kí tự khác: %d", other);
}
```

Ví dụ: Tìm tổng S = 1 + 1 /3 + 1/5 + . .+ 1/(2n-1) với độ chính xác e (1/n >=e);

```
#include    <stdio.h>
#include    <conio.h>
void main(void){
    int     i =1;
    loat    s=0, epsilon;
    clrscr();
    printf("\n Nhập độ chính xác epsilon="); scanf("%f",&epsilon);
    while( ( (float) 1 / (float i) )>=epsilon) {
            s+=(float) 1 / (float) i;
            i+=2;
    }
    printf("\n Tổng s=%6.2f", s);
    getch();
}
```

Ví dụ: Tính e^x theo công thức xấp xỉ chuỗi Taylor với e = x^n/n!.

$e^x = 1 + x/1! + x^2/2! + x^3/3! + . . . + x^n/n!$

```
#include    <stdio.h>
#include    <conio.h>
void main(void){
    float   e_mu_x, epsilon, x, t;
    int     n; clrscr();
    printf("\n Nhập x="); scanf("%f", &x);
    printf("\n Nhập độ chính xác epsilon="); scanf("%f", &epsilon);
    e_mu_x = 1; n = 1; t = x;
    while ( t >=epsilon) {
            e_mu_x += t ;
            n++; t = t * (x/n);
    }
    printf("\n e mũ %6.3f = %6.3f", x, e_mu_x);
    getch();
}
```

3.6. Vòng lặp không xác định do . . while

Cú pháp:

```
do {
```

PTIT

```
    câu_lệnh;
} while(biểu_thức);
```

Thực hiện câu lệnh trong khi biểu_thức vẫn còn đúng, nếu biểu thức có giá trị sai, điều khiển chương trình chuyển về lệnh kế tiếp ngay sau while(biểu_thức).

Ví dụ: Viết chương trình xây dựng tập thao tác cộng, trừ, nhân, chia, lấy phần dư của hai số nguyên a,b.

```
#include    <stdio.h>
#include    <conio.h>
#include    <dos.h>         /* sử dụng hàm delay()*/
#define     F1      59      /* định nghĩa phím F1 */
#define     F2      60      /* định nghĩa phím F2 */
#define     F3      61      /* định nghĩa phím F3 */
#define     F4      62      /* định nghĩa phím F4 */
#define     F5      63      /* định nghĩa phím F5 */
#define     F6      64      /* định nghĩa phím F6 */
#define     F10     68      /* định nghĩa phím F10 */
void main(void){
    int a, b, control=0; char key;
    clrscr();
    do {
        printf("\n Tập thao tác với hai số nguyên a, b");
        printf("\n F1- Nhập hai số nguyên a,b");
        printf("\n F2-Tổng hai số nguyên");
        printf("\n F3-Hiệu hai số nguyên");
        printf("\n F4- Tích hai số nguyên");
        printf("\n F5- Thương hai số nguyên");
        printf("\n F6- Modul hai số nguyên");
        printf("\n F10- Trở về");
        key = getch();
        switch(key) {
            case F1:
                printf("\n Nhập a="); scanf("%d", &a);
                printf("\n Nhập b="); scanf("%d", &b);
                control =1; break;
            case F2:
                if( control !=0 )
                    printf("\n Tổng a + b =%d", a+b);
                break;
            case F3:
                if( control !=0 )
                    printf("\n Hiệu a - b =%d", a - b);
```

PTIT

```
                    break;
                case F4:
                    if( control !=0 )
                        printf("\n Tích a * b =%d", a * b);
                    break;
                case F5:
                    if( control !=0 )
                     printf("\nThương a*b=%6.2f",(float)a/ (float)b);
                    break;
            }
            clrscr();
        } while(key!=F10);
    }
```

3.7. Lệnh break và lệnh Continue

Lệnh break cho phép ra khỏi các chu trình với các toán tử for, while và switch.

Ví dụ 1: Tính tổng S =1+2+3+4+5

```
s=0;
i=1;
for(i=1;i<=10;i++)
 {if(i==6) break;
   s=s+i;
}
```

Lệnh continue dùng để bắt đầu một vòng mới của chu trình chứa nó. Lệnh continue trong trường hợp for điều khiển được chuyển về bước khởi đầu lại. Còn trong while, do...while quyền điều khiển chương trình được chuyển đến bước kiểm tra điều kiện.

Ví dụ 2: Tính tổng S =1+2+3+4+5+7+8+9+10

```
s=0;
i=1;
for(i=1;i<=10;i++)
{if(i==6) continue;
   s=s+i;
}
```

ở ví dụ 1 S= 1+2+3+4+5 = 15

ở ví dụ 2 S= 1+2+3+4+5+7+8+9+10 = 49

4. HÀM VÀ PHẠM VI HOẠT ĐỘNG CỦA BIẾN

4.1. Tính chất của hàm

Hàm (function) hay nói đúng hơn là chương trình con (sub_program) chia cắt các nhiệm vụ tính toán lớn thành các công việc nhỏ hơn và có thể sử dụng nó ở mọi lúc trong chương trình, đồng thời hàm cũng có thể được cung cấp cho nhiều người khác sử dụng dưới dạng thư viện mà không cần phải bắt đầu xây dựng lại từ đầu. Các hàm thích hợp còn